Roll No.	Subject Code	Assignment No.
2650026	CS69101	Lab 8

Implementation

Implement a Binary Min Heap using an array.

Your implementation must support:

1. Construct a Min Heap from the given unsorted array using bottom-up heap construction.
2. Insert a new element into the Min Heap.
3. Find the minimum element.
4. Extract-Min operation.
5. Delete an arbitrary element
6. Decrease-Key operation for a specified element.
7. Display the heap after every operation.

In [1]:

```
import sys
from io import TextIOWrapper
from typing import Self, Any, Callable, TypeVar, Generic, Optional
import random
import sys
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.pylab import figure

sys.setrecursionlimit(100000) # Fix to avoid RecursionError

prep_arr = [random.randint(1, (i+1)*10) for i in range(100000)] # Random Array of 100000 Elements
# prep_arr.sort() # Sorted Array for Worst Case Scenario
print(prep_arr[:25])
print(len(prep_arr))
```

```
# Write the array to a file
f = open("./2650026_lab8_data/input0.txt", "w")
for i in prep_arr:
    f.write(f"{i}, ")
f.write("\n")
f.close()

del prep_arr
del f

iterations = 0
swaps = 0
PRINT_DEBUG = False
```

```
[9, 2, 25, 16, 36, 46, 64, 22, 81, 5, 95, 112, 107, 84, 38, 116, 105, 30, 105, 108, 94, 192, 58, 188, 173]
100000
```

In [2]:

```
def get_random_sample(arr, n):
    if n > len(arr):
        raise ValueError("n cannot exceed the array length")

    start = random.randint(0, len(arr) - n)
    return arr[start : start + n]
```

In [3]:

```
class MinHeap:
    def __init__(self, arr: list[int] | None = None):
        if arr is None:
            self.arr = []
        else:
            self.arr = arr
        self.size = len(self.arr)
        self.bottom_up_build_min_heap()

    def _parent_index(self, i: int) -> int:
        return (i - 1) // 2

    def _left_child_index(self, i: int) -> int:
        return 2 * i + 1

    def _right_child_index(self, i: int) -> int:
        return 2 * i + 2
```

```
    def get_min(self) -> int:
        if self.size == 0:
            raise IndexError("[ERROR] Getting minimum from an empty heap.")
        return self.arr[0]

    def min_heapify(self, i: int):
        global iterations, swaps
        iterations += 1

        left = self._left_child_index(i)
        right = self._right_child_index(i)
        # Find the index of the smallest element among the current node and its children
        # smallest = i

        if left < self.size and self.arr[left] < self.arr[i]:
            smallest = left
        else:
            smallest = i

        if right < self.size and self.arr[right] < self.arr[smallest]:
            smallest = right

        if smallest != i:
            self.arr[i], self.arr[smallest] = self.arr[smallest], self.arr[i]
            swaps += 1
            if PRINT_DEBUG:
                print(f"Swapped {self.arr[i]} and {self.arr[smallest]} at indices {i} and {smallest}")
            self.min_heapify(smallest)

    def bottom_up_build_min_heap(self, arr: list[int] | None = None):
        if arr is not None:
            self.arr = arr
            self.size = len(self.arr)
        for i in range(self.size // 2 - 1, -1, -1):
            self.min_heapify(i)

    def top_down_build_min_heap(self, arr: list[int] | None = None):
        if arr is not None:
            self.arr = arr
            self.size = len(self.arr)
        for i in range(self.size):
```

```
            self.decrease_key(i, self.arr[i])

    def extract_min(self) -> int:
        if self.size == 0:
            raise IndexError("[ERROR] Extracting from an empty heap.")
        min_value = self.arr[0]
        self.arr[0] = self.arr[self.size - 1]
        self.size -= 1
        self.arr.pop()  # Remove the last element
        self.min_heapify(0)
        return min_value

    def min_heap_insert(self, value: int):
        global swaps

        self.arr.append(value)
        self.size += 1
        i = self.size - 1
        while i > 0 and self.arr[self._parent_index(i)] > self.arr[i]:
            parent = self._parent_index(i)
            self.arr[i], self.arr[parent] = self.arr[parent], self.arr[i]
            swaps += 1
            if PRINT_DEBUG:
                print(f"Swapped {self.arr[i]} and {self.arr[parent]} at indices {i} and {parent}")
            i = parent

    # def min_heap_delete(self, i: int):
    #     if i<0 or i>=self.size:
    #         raise IndexError("[ERROR] Index out of bounds.")
    #     last_value = self.arr[self.size - 1]
    #     self.size -= 1
    #     if i < self.size:
    #         self.arr[i] = last_value
    #         self.min_heapify(i)
        # self.decrease_key(i, int(float('-inf')))  # Decrease the key to negative infinity
        # self.extract_min()  # Extract the minimum element (which is now at index 0)

    def min_heap_delete(self, i: int):
        global swaps

        if i < 0 or i >= self.size:
            raise IndexError("[ERROR] Index out of bounds.")
```

```
        self.size -= 1
        if i == self.size:
            return
        self.arr[i] = self.arr[self.size]
        if i > 0 and self.arr[i] < self.arr[self._parent_index(i)]:
            while i > 0:
                parent = self._parent_index(i)
                if self.arr[parent] <= self.arr[i]:
                    break
                self.arr[i], self.arr[parent] = self.arr[parent], self.arr[i]
                swaps += 1
                i = parent
        else:
            self.min_heapify(i)

    def decrease_key(self, i: int, new_value: int):
        global swaps

        if new_value > self.arr[i]:
            raise ValueError("[ERROR] New value is greater than the current value.")
        self.arr[i] = new_value
        while i > 0 and self.arr[self._parent_index(i)] > self.arr[i]:
            parent = self._parent_index(i)
            self.arr[i], self.arr[parent] = self.arr[parent], self.arr[i]
            swaps += 1
            if PRINT_DEBUG:
                print(f"Swapped {self.arr[i]} and {self.arr[parent]} at indices {i} and {parent}")
            i = parent

    def heap_search(self, i: int = 0, value: int = 0) -> int:
        global iterations
        iterations += 1

        if self.size == 0:
            return -1
        if i >= self.size or self.arr[i] > value:
            return -1
        if self.arr[i] == value:
            return i
        left_res = self.heap_search(self._left_child_index(i), value)
        if left_res != -1:
            return left_res
```

```
        return self.heap_search(self._right_child_index(i), value)

    def __str__(self) -> str:
        return str(self.arr[:self.size])

    def __repr__(self) -> str:
        return f"MinHeap({self.arr[:self.size]})"
```

Given an unsorted array of N distinct integers, design an algorithm to identify the Kth smallest element using a Min Heap.

In [4]:

```
def kth_smallest_min_heap(self, N, k: int) -> int:
    if k < 1 or k > N:
        raise ValueError("[ERROR] k is out of bounds.")
    min_heap = MinHeap(self.arr[:N])
    kth_smallest = None
    for _ in range(1, k):
        kth_smallest = min_heap.extract_min()
    return min_heap.get_min() if kth_smallest is None else kth_smallest
```

Testing Implementation for above stated operations

In [5]:

```
# Read a set of integer elements from an input file (input.txt)
arr0 = []
data0 = open("./2650026_lab8_data/input.txt", 'r')
try:
    for line in data0.readlines():
        line_data = line.rstrip().rstrip(',').replace(',',' ').split() #using rstrip to remove the \n
        for data in line_data:
            arr0.append(int(data))
        data0.close()
except Exception as e:
    data0.close()
    sys.exit("[ERROR] Invalid data" + str(e))
```

In [6]:
```
minheap0 = MinHeap()
MinHeap.bottom_up_build_min_heap(minheap0, arr0)
```

In [7]:
```
print(minheap0)
```

```
[25, 50, 76, 67, 55, 210, 90, 77, 69, 420, 70, 911, 348, 108, 768, 1080, 720]
```

In [8]:
```
MinHeap.min_heap_insert(minheap0, 101)
print(minheap0)
```

```
[25, 50, 76, 67, 55, 210, 90, 77, 69, 420, 70, 911, 348, 108, 768, 1080, 720, 101]
```

In [9]:
```
minheap0.get_min()
```

Out[9]: 25

In [10]:
```
minheap0.extract_min()
```

Out[10]: 25

In [11]:
```
print(f"MinHeap after extract_min: {minheap0}")
```

```
MinHeap after extract_min: [50, 55, 76, 67, 70, 210, 90, 77, 69, 420, 101, 911, 348, 108, 768, 1080, 720]
```

In [12]:
```
idx_del = minheap0.heap_search(value=67)
if idx_del != -1:
    minheap0.min_heap_delete(idx_del)
print(f"MinHeap after deleting 67: {minheap0}")
```

```
MinHeap after deleting 67: [50, 55, 76, 69, 70, 210, 90, 77, 720, 420, 101, 911, 348, 108, 768, 1080]
```

In [13]:
```
minheap0.decrease_key(6, 67)
```

In [14]:
```
print(f"MinHeap after decreasing key at index 6 to 67: {minheap0}")
```

```
MinHeap after decreasing key at index 6 to 67: [50, 55, 67, 69, 70, 210, 76, 77, 720, 420, 101, 911, 348, 108, 768,
1080]
```

Performance Evaluation

Evaluate the performance of your Min Heap implementation for the following input sizes: N = 1,000; 5,000; 10,000; 50,000; 100,000.

In [ ]:
```

```

Conduct experiments using:

1. Randomly generated input.
2. Ascending sorted input.
3. Descending sorted input.